Bash Scripting - One-In-All Handbook (Deep Guide)

Bash Scripting - One-In-All Handbook (Deep Guide)

This guide is a comprehensive reference and quick reminder for Bash scripting on Linux. It covers core concepts, variables, operators, conditionals, loops, arrays, user input, script arguments, redirection and pipes, exit status and error codes, debugging, file and directory handling, text processing, cron jobs, comments, functions, signals and traps, here-docs, quoting and expansion, portability, and more. All examples are plain-text and code, without tables or boxes, to keep it clean and printable.

Bash Scripting - One-In-All Handbook (Deep Guide)

1) What is a Bash script?

A Bash script is a plain text file containing commands executed by the Bash shell (/bin/bash). Scripts automate tasks such as backups, deployments, ETL jobs, checks, and system administration. To run a script: make it executable and execute it from the shell.

```
#!/usr/bin/env bash
echo "Hello from Bash"
ls -la
exit 0
```

Professional skeleton

```
#!/usr/bin/env bash
set -Eeuo pipefail
IFS=$'\n\t'

log(){ printf '[%s] %s\n' "$(date +%F\ %T)" "$*"; }
die(){ log "ERROR: $*"; exit 1; }
main(){
  log "Starting"
  # your logic here
}
main "$@"
```

2) How Bash runs commands and scripts

Bash reads a line, performs expansions (brace, tilde, parameter, command substitution, arithmetic, globbing), does word splitting, then executes the command. Path resolution uses PATH for non-absolute names. Sourcing (.) executes in current shell; running executes in a child shell process.

```
# Run in a new process
./myscript.sh
# Source into current shell
. ./myscript.sh   # or: source myscript.sh
```

Bash Scripting - One-In-All Handbook (Deep Guide)

3) Variables in Bash (deep)

3.1 Basics and assignment

Variables are untyped strings by default. No spaces around =. Use quotes to preserve spaces and avoid globbing.

```
name='Alice'
greeting="Hello $name"
echo "$greeting"
```

3.2 Numeric variables and arithmetic contexts

Use arithmetic expansions $((...)) or arithmetic command ((...)) for integer math. Optional declare -i enforces integer behavior on a variable.

```
a=10; b=3
sum=$(( a + b ))
((a+=5))
declare -i n=7; n+=2; echo "$n"
```

3.3 Arrays (indexed) meaning, definition, purpose, types

Indexed arrays are ordered lists accessed by numeric index starting at 0. Purpose: store lists such as filenames, IDs, or options. Type: 'indexed array'.

```
declare -a nums=(10 20 30)
echo "First: ${nums[0]}"
echo "All:   ${nums[@]}"
echo "Idxs:  ${!nums[@]}"
echo "Len:   ${#nums[@]}"
nums+=(40 50)
unset 'nums[1]'
```

3.4 Arrays (associative) meaning, definition, purpose, types

Associative arrays are key-value maps indexed by strings. Purpose: store settings, counters by name, or lookups. Enable with declare -A.

```
declare -A ages=([alice]=30 [bob]=28)
echo "Alice: ${ages[alice]}"
for k in "${!ages[@]}"; do echo "$k => ${ages[$k]}"; done
```

3.5 Environment variables, exported variables, and local

Exported variables become part of the environment of child processes. Local variables exist in current shell. Use local inside functions to limit scope.

```
export PATH="$HOME/bin:$PATH"
APP_HOME=/opt/app
myfunc(){ local tmp=42; echo "$tmp"; }
myfunc; echo "Global PATH: $PATH"
```

3.6 Positional and special parameters

```
echo "$0"      # script name
echo "$1"      # first argument
echo "$#"      # number of args
echo "$@"      # all args, split as words
echo "$*"      # all args as one word (joined by IFS)
```

```
echo "$?"      # exit status of last command
echo "$$"      # PID of current shell
echo "$!"      # PID of last background job
```

3.7 Quoting rules and best practices

Double quotes expand variables and command substitutions but prevent word splitting and globbing. Single quotes prevent all expansions. Always quote variables in tests and path operations.

```
f="/tmp/a b.txt"
rm "$f"   # safe
echo "$var"   # safe expansion
```

3.8 Parameter expansion techniques

```
v=${v:-default}
v=${v:=default}
: ${REQUIRED:?missing var}
len=${#v}
sub=${v:2:3}
x=${v/foo/bar}
x=${v//foo/bar}
x=${v#prefix}
x=${v%suffix}
```

4) Operators in Bash and why to use them

4.1 Arithmetic operators and contexts

```
(( a=5+3 ))   # + - * / % **
(( a+=2, b=a++ ))
result=$(( (a*b) % 7 ))
```

4.2 Numeric comparison: -eq vs = and (( ))

Use -eq, -ne, -lt, -le, -gt, -ge for numeric comparisons inside [ ] (test). Use = or != for string comparisons. In arithmetic context (( )), you can write a < b, a == b, etc. Reason: Bash treats values as strings unless you use numeric operators or arithmetic context. Using = compares text, not numbers, so 10 and 2 are just strings. Use -gt to compare them numerically.

```
a=10; b=2
[ "$a" -gt "$b" ] && echo "10 is greater"
[ "$a" = "$b" ] && echo "equal strings" || echo "different strings"
if (( a > b )); then echo "numeric compare via (( ))"; fi
```

4.3 String operators

```
[ "$s" = "hello" ]
[ "$s" != "world" ]
[ -n "$s" ]  # non empty
[ -z "$t" ]  # empty
[[ "$s" < "zoo" ]]  # lexicographic in [[ ]]
```

4.4 Logical operators

```
[ -f file ] && echo exists || echo missing
if [[ -d src && -x build.sh ]]; then ./build.sh; fi
! false && echo ok
```

4.5 File test operators

```
[ -e path ]  # exists
[ -f file ]  # regular file
[ -d dir  ]  # directory
[ -s file ]  # size > 0
[ -r file ]  # readable
[ -w file ]  # writable
[ -x file ]  # executable
[ -L link ]  # symlink
[ file1 -nt file2 ]  # newer than
[ file1 -ot file2 ]  # older than
```

4.6 Regex matching in [[ ]]

```
s="abc123"
if [[ $s =~ ^[a-z]+[0-9]+$ ]]; then echo match; fi
```

5) Conditional statements: purpose and usage

Conditionals choose code paths based on tests. Bash offers if/elif/else and case/esac.

5.1 if/elif/else

```
read -p "Age: " age
if (( age >= 18 )); then
  echo Adult
elif (( age >= 13 )); then
  echo Teen
else
  echo Child
fi
```

5.2 case/esac

```
read -p "Answer [y/n]: " a
case "$a" in
  y|Y|yes) echo Proceed;;
  n|N|no)  echo Abort;;
  *)       echo Invalid;;
esac
```

6) Loop statements: purpose, usage, types

6.1 for-in over a list

```
for name in Alice Bob Carol; do
  echo "Hello $name"
done
```

6.2 C-style for loop

```
for ((i=0; i<5; i++)); do
  echo "$i"
done
```

6.3 while loop

```
i=1
while (( i <= 3 )); do
  echo "$i"; ((i++))
done
```

6.4 until loop

```
x=0
until (( x == 3 )); do
  echo "$x"; ((x++))
done
```

6.5 break and continue

```
for i in {1..10}; do
  (( i==3 )) && continue
  (( i==6 )) && break
  echo "$i"
done
```

6.6 Reading files safely in loops

```
while IFS= read -r line; do
  printf '%s\n' "$line"
done < input.txt
```

6.7 select (menus)

```
PS3='Pick a fruit: '
select f in apple banana cherry; do
  echo "You picked $f"; break
done
```

7) User input: types, meaning, definition, defining user input

User input in Bash is primarily collected using read, select, and command line flags via getopts.

7.1 read: prompt and read a line

```
read -p "Enter name: " name
echo "Hello $name"
```

7.2 read with timeout, silent, arrays

```
read -t 5 -p "Answer within 5s: " ans || echo timeout
read -s -p "Password: " pass; echo
read -a items -p "Enter items: "  # fills an indexed array
```

7.3 select: menu input

```
select opt in start stop status; do
  echo "You chose: $opt"; break
done
```

7.4 getopts: parse -f style flags

```
while getopts ":u:p:v" opt; do
  case $opt in
    u) user=$OPTARG;;
    p) pass=$OPTARG;;
    v) verbose=1;;
    :) echo "Option -$OPTARG requires an argument"; exit 2;;
    \?) echo "Unknown option -$OPTARG"; exit 2;;
  esac
done
shift $((OPTIND-1))
```

8) Script arguments: meaning and usage

Arguments are passed after the script name and accessed via $1, $2, ..., $@, $#. Use shift to consume.

```
# myscript.sh file1 file2
echo "Script: $0"
echo "Count:  $#"
for a in "$@"; do echo "Arg: $a"; done
while [[ $# -gt 0 ]]; do
  echo "Processing: $1"; shift
done
```

9) Redirections and pipes

Redirect stdout, stderr, and use pipelines to chain commands.

```
cmd > out.txt        # stdout to file (truncate)
cmd >> out.txt       # append
cmd 2> err.txt       # stderr to file
cmd > out 2>&1       # both to out
cmd 2>&1 | tee log   # stream both into pipe
grep foo file | sort | uniq -c | sort -nr
```

Here-strings and here-docs

```
cat <<< "single line"
cat <<'EOF'
multi line text
EOF
```

10) Exit status and error codes

Every command returns an exit code in $?. 0 means success, non-zero indicates failure. Use set -e to exit on first error in simple scripts, or handle errors explicitly with || and if.

```
false; echo $?      # prints non-zero
true;  echo $?      # prints 0
cp src dst || { echo "copy failed"; exit 1; }
```

Custom exit codes

```
die(){ echo "ERROR: $*" >&2; exit 2; }
[[ -f "$1" ]] || die "Missing file: $1"
```

11) Debugging commands and techniques

Trace execution with set -x and PS4, validate with shellcheck, inspect variables and functions, use trap to capture errors and line numbers.

```
set -x            # enable xtrace
set +x            # disable xtrace
PS4='+${BASH_SOURCE}:${LINENO}:${FUNCNAME[0]}: '
set -x
trap 'echo "ERR at ${BASH_SOURCE}:${LINENO} (status=$?)"' ERR
```

Useful checks

```
type -a cmd        # where is a command resolved
declare -p var     # show attributes and value
compgen -c | wc -l # list commands available
```

12) File and directory handling

Create, list, find, copy, move, delete safely. Always quote paths.

```
mkdir -p "$dst"
cp -r "$src" "$dst"   # recursive copy
mv -- "$old" "$new"
rm -f -- "$file"
rm -rf -- "$dir"
```

find basics

```
find /var/log -type f -name '*.log' -size +10M -mtime +7 -print
find . -maxdepth 1 -type f -print0 | xargs -0 ls -l
```

Permissions and modes

```
chmod 654 file    # rw-r-xr--
chmod 000 file    # no permissions
chown user:group file
umask 022         # default mode mask
```

13) Text processing commands (quick remind)

grep

```
grep -n "pattern" file
grep -r "TODO" src/
```

sed

```
sed -n '1,10p' file
sed 's/foo/bar/g' file
sed -E 's/^([0-9]+),([a-z]+)/\2 \1/' data.csv
```

awk

```
awk -F, '{sum+=$3} END{print sum}' data.csv
awk '{print NR ":" $0}' file
```

cut, tr, sort, uniq, paste, join

```
cut -d, -f2 file
tr '[:lower:]' '[:upper:]' < file
sort file | uniq -c | sort -nr
paste f1 f2
join -1 1 -2 1 f1 f2
```

xargs and parallelism

```
printf '%s\n' file1 file2 | xargs -n1 -P4 gzip
```

14) Cron jobs: scheduling scripts

Use crontab -e to edit per-user crontab. Format: m h dom mon dow command. Use absolute paths and environment variables as needed.

```
crontab -e
# Every day at 02:30
30 2 * * * /home/user/backup.sh >> /home/user/backup.log 2>&1
# Every 5 minutes
*/5 * * * * /usr/local/bin/health-check
```

Systemd timers (alternative)

```
# On systemd systems consider timers for reliability
# Create .service and .timer units
```

15) Comments and coding style

Use # for comments. Keep scripts readable, modular, and safe. Quote variables, set -Eeuo pipefail, use functions, and return explicit exit codes.

```
# This script deploys the application
set -Eeuo pipefail
# TODO: add retries
```

16) Functions: definition and use

Functions group commands and can accept arguments via $1, $2 or named locals. Return values are exit codes or printed output captured with command substitution.

```
greet(){
  local who=${1:-world}
  echo "Hello, $who"
}
msg=$(greet Alice)
echo "$msg"
```

17) Signals and traps

Trap signals to cleanup temporary files or stop gracefully.

```
tmp=$(mktemp)
cleanup(){ rm -f "$tmp"; }
trap cleanup EXIT INT TERM
echo data > "$tmp"
sleep 2
echo done
```

18) Globbing and brace expansion

Globbing expands wildcards like *. Brace expansion generates strings before globbing.

```
echo src/*.c
echo file_{a,b,c}.txt
```

19) Subshells vs current shell; sourcing vs executing

Parentheses create a subshell; changes inside do not affect the parent shell. Sourcing runs in current shell.

```
(cd /tmp; echo "Here: $(pwd)")
echo "Back: $(pwd)"
. ./env.sh  # source into current shell
```

20) Portability notes: POSIX sh vs Bash

For portability, avoid Bash-only features (arrays, [[ ]], =() maps, process substitution) and target /bin/sh. If you need Bash features, use shebang #!/usr/bin/env bash and document Bash requirement.

21) Script templates you can reuse

21.1 Minimal strict-mode script

```
#!/usr/bin/env bash
set -Eeuo pipefail
IFS=$'\n\t'
main(){
  echo "Hello"
}
main "$@"
```

21.2 CLI script with getopts

```
#!/usr/bin/env bash
set -Eeuo pipefail
show_help(){ echo "Usage: $0 -i INPUT -o OUTPUT [-v]"; }
input= output= verbose=0
while getopts ":i:o:vh" opt; do
  case $opt in
    i) input=$OPTARG;;
    o) output=$OPTARG;;
    v) verbose=1;;
    h) show_help; exit 0;;
    :) echo "Missing arg for -$OPTARG"; exit 2;;
    \?) echo "Unknown -$OPTARG"; exit 2;;
  esac
done
shift $((OPTIND-1))
[[ -n "$input" && -n "$output" ]] || { show_help; exit 2; }
echo "Processing $input -> $output"
```

22) Quick reminder lines

Use [[ ]] for safer string tests; use (( )) for numeric logic. Quote paths. Prefer arrays over parsing ls. Use mapfile/readarray to read lines into arrays. Prefer grep -E for regex. Use pipefail to fail pipelines. Prefer printf over echo for portability. Avoid for i in $(cmd) because of word splitting; use while read -r. Check exit codes, log steps, and always test scripts with set -x in a non-production environment.